Hackathon Registration Service

Scalability & Future Roadmap Blueprint

Engineering-Grade Performance Analysis · Architecture Design · Nationwide Deployment Plan

CURRENT CAPACITY	TARGET CAPACITY	SCALE-UP FACTOR	PRODUCTION READINESS
~100 users	30,000 users	300×	NOT READY

Document Type	Engineering Scalability Blueprint
Application	Spheronix Hackathon Registration Service
Analysis Date	2026-08-11
Testing Tool	k6 Load Testing Framework
Target Scale	30,000 Concurrent Users / 5,000 RPS
Status	NOT READY — Architectural Changes Required

1. Executive Summary

The Spheronix Hackathon Registration Service is a well-architected FastAPI application backed by an asynchronous MongoDB driver (Motor). Under minimal load it delivers excellent baseline performance — p95 response times of **50.68 ms** at 5 concurrent virtual users. However, rigorous k6 load testing revealed that the single-node deployment **collapses entirely at approximately 150 concurrent users**, producing a 54.22% error rate at 250 VUs due to TCP connection refusals from the overwhelmed Uvicorn process.

The application's **measured sustainable capacity** is ~100 concurrent users producing 45–50 RPS. The **nationwide deployment target** is 30,000 concurrent users requiring a sustained 5,000 RPS with zero tolerance for downtime during registration deadline events. This represents a **300× gap in concurrency** and a **100× gap in throughput**.

PRODUCTION READINESS VERDICT

NOT READY FOR NATIONWIDE DEPLOYMENT

The application requires fundamental architectural changes before it can serve 30,000 concurrent users. The business logic is correct — the infrastructure must change.

The path forward is well-defined: **offload synchronous blocking operations** to Celery background workers, **distribute shared state** to Redis, **containerise** the application and deploy across a Kubernetes cluster with auto-scaling, and **serve static assets through a CDN**. With these changes, the same business logic will reliably serve 30,000 concurrent users with headroom to spare.

2. Present State — Architecture & Performance

2.1 Current Architecture

The application is a **Python FastAPI** service running on a single Uvicorn process. It connects to MongoDB using the Motor async driver. SlowAPI provides in-memory rate limiting. Static HTML/CSS/JS files are served directly from the FastAPI process via StaticFiles mount. All operations — file parsing, email delivery, database writes — execute synchronously within the single async event loop.

Component	Technology	Current Status
Web Framework	FastAPI + Uvicorn (single process)	■ Single point of failure
Process Manager	python main.py (no Gunicorn)	■ No multi-worker support
Database	MongoDB (Motor async)	✓ Well indexed, performs well
Rate Limiting	SlowAPI (in-memory)	✗ Breaks with multiple instances
OTP Storage	In-memory dict (otp_store)	✗ Not shareable across instances
File Parsing	Pandas (synchronous, blocking)	✗ Blocks the event loop
Email Delivery	SMTP (synchronous, blocking)	✗ Blocks the event loop
Image Storage	Base64 in MongoDB documents	■ Inflates document size
Static Assets	Served by FastAPI (StaticFiles)	■ Wastes application resources
Caching	None	✗ Every request hits MongoDB
Observability	Basic Uvicorn stdout logs	■ No metrics or alerting

2.2 Measured Performance Results

All metrics below were measured using k6 against a locally running Uvicorn instance connected to a live MongoDB database. No values are estimated.

Metric	Measured Value	Assessment
Sustainable concurrent users	~100	Critical — far below target
Sustainable RPS	~45–50 RPS	Critical — 100× below target
Peak RPS observed	~60 RPS	Critical
p95 Latency @ 5 VUs (baseline)	50.68 ms	✓ Excellent
p95 Latency @ 250 VUs (success only)	35.7 ms	N/A — 54% failed
Error rate @ 250 VUs	54.22%	✗ Server failure
Excel upload p95 (5 VUs, small file)	100.73 ms	■ Will worsen at scale
Server failure point	~150 concurrent users	✗ Critical threshold
Max response time observed	394.76 ms	■ Outliers present

2.3 Identified Bottlenecks

■ CRITICAL

Excel/CSV Parsing Blocks Event Loop

pd.read_excel() is synchronous. It occupies the main thread for 200 ms to 5 seconds per file. During this time no other coroutine can execute — health checks, logins, and registrations all queue and time out.

■ CRITICAL

SMTP Email Delivery is Synchronous

The OTP endpoint waits for the SMTP server to respond before returning. This blocks the request thread for 1–5 seconds per OTP, limiting throughput to ~0.2 OTP requests per second per worker under high load.

■ HIGH

Rate Limiting State is Not Distributed

SlowAPI uses an in-memory counter store. With multiple nodes, each pod maintains independent counters. A user can exceed rate limits by a factor equal to the number of pods, and limits cannot be enforced globally.

■ HIGH

OTP Storage is Not Distributed

OTPs are stored in an in-memory Python dictionary (otp_store). With a load balancer, a user's OTP verification request may route to a different pod than the one that generated the OTP, causing all verifications to fail.

■ MEDIUM

Base64 Images Stored in MongoDB

Payment proof images up to 30 KB are stored as Base64 strings directly in registration documents. This inflates each document by up to 40 KB, slowing bulk reads and administrative exports significantly at scale.

■ MEDIUM

MongoDB Connection Pool Unconfigured

Motor's default connection pool settings are not tuned for a multi-pod deployment. With 50 pods and 4 workers each, the cluster could attempt 200+ concurrent connections to MongoDB without explicit pool limits.

2.4 Scalability Gap Analysis

Dimension	Current	Required	Gap
Concurrent users	~100	30,000	300×
Requests per second	~50 RPS	5,000 RPS	100×
Failure tolerance	None (single node)	Multi-zone HA	Fundamental redesign
Horizontal scaling	Not possible (stateful)	Fully stateless	Architecture change
Background processing	None	Celery worker fleet	New component
Distributed caching	None	Redis cluster	New component

3. Target Architecture for 30,000 Users

3.1 Architecture Layers

The target architecture replaces the single Uvicorn process with a six-layer cloud-native system. Each layer is independently scalable and has no single point of failure. The layers and their responsibilities are described below.

Layer 1 — Edge	**Cloudflare / AWS CloudFront CDN + WAF** Absorbs 60–70% of all traffic by serving static assets (HTML, CSS, JS, images) from edge nodes globally. Blocks malicious bots, DDoS attacks, and SQL injection before they reach the origin.
Layer 2 — Load Balancer	**AWS Application Load Balancer (ALB)** Distributes API traffic across all healthy Kubernetes pods. Performs health checks on /api/health every 5 seconds. Provides cross-availability-zone redundancy and connection draining for zero-downtime pod restarts.
Layer 3 — Application	**FastAPI on Kubernetes (5–50 pods, HPA)** Runs stateless FastAPI instances managed by Gunicorn with 4 Uvicorn workers per pod. Kubernetes HPA automatically scales pods from 5 to 50 based on CPU utilisation. All shared state is externalised to Redis.
Layer 4 — Task Queue	**Celery Workers + Redis Broker** Processes all CPU-bound and I/O-blocking tasks asynchronously: Excel parsing, email OTP delivery, PDF generation, and export operations. Workers scale independently from web pods using KEDA based on queue depth.
Layer 5 — Data	**MongoDB Atlas M40 + Redis Cluster** MongoDB Atlas provides a 3-node replica set with automatic failover, backups, and monitoring. Redis serves as the shared cache, rate limit store, OTP store, and Celery broker across all pods.
Layer 6 — Storage	**AWS S3 (Two Buckets)** hackathon-static (public, CDN-served) for frontend assets. hackathon-uploads (private) for Excel files and payment proof images. Eliminates binary data from MongoDB documents.

3.2 Projected Capacity After Full Implementation

Pods Active	Workers / Pod	Estimated Users	Estimated Peak RPS	Scenario
5 (minimum)	4	~2,000	~500 RPS	Off-peak / idle
15	4	~7,500	~1,500 RPS	Normal weekday traffic
25	4	~12,500	~2,500 RPS	Heavy day
40	4	~25,000	~4,500 RPS	Deadline day peak
50 (maximum)	4	~30,000+	~5,500 RPS	Maximum surge

4. Implementation Plan — Critical Code Changes

4.1 Excel Upload: Synchronous → Asynchronous

The most critical change. Replace the blocking pd.read_excel() call in /api/dashboard/upload-data with an immediate file save to S3 followed by a Celery task enqueue. The endpoint returns a 202 Accepted response in ~60 ms instead of blocking for up to 5 seconds.

CURRENT CODE (blocks event loop)

```
# pd.read_excel() is synchronous — freezes all requests during parse
df = pd.read_excel(io.BytesIO(content))
validation_result = ExcelValidator.validate(json_data)
```

REQUIRED CODE (returns in ~60 ms)

```
# Save to S3, enqueue Celery task, return 202 immediately
s3_key = f'uploads/{user_id}/{uuid.uuid4()}.xlsx'
await s3_client.put_object(Bucket='hackathon-uploads', Key=s3_key, Body=content)
task = celery_app.send_task('tasks.process_excel', args=[s3_key, user_id])
return JSONResponse(status_code=202, content={'job_id': task.id})
```

4.2 OTP Email: Blocking → Non-Blocking

Store OTPs in Redis with a 10-minute TTL and enqueue email delivery as a Celery task. The endpoint response time drops from 1,000–5,000 ms (SMTP round-trip) to under 10 ms. This also fixes the horizontal scaling bug where OTPs generated on Pod A are invisible to Pod B.

4.3 Rate Limiting: In-Memory → Distributed Redis

ONE-LINE FIX (add storage_uri parameter to Limiter)

```
# Before (in-memory — does not work across multiple pods):
limiter = Limiter(key_func=get_remote_address)

# After (Redis-backed — enforced globally across all 50 pods):
limiter = Limiter(
    key_func=get_remote_address,
    storage_uri=settings.redis_url   # e.g. redis://redis-cluster:6379
)
```

4.4 Payment Proof: Base64 in MongoDB → AWS S3

Replace the Base64 encoding and MongoDB storage with an S3 upload, storing only the S3 object key (a ~60-character string) in the database document. This keeps MongoDB documents lightweight (under 2 KB) and makes bulk reads and administrative exports orders of magnitude faster at 30,000 registrations.

4.5 Kubernetes Deployment Configuration

Dockerfile — Production Entry Point

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["gunicorn", "app.main:app",
     "--workers", "4",
     "--worker-class", "uvicorn.workers.UvicornWorker",
     "--bind", "0.0.0.0:8000",
     "--graceful-timeout", "30"]
```

5. Future State & Long-Term Roadmap

Beyond the immediate scaling fix, the following roadmap describes the evolution of the Spheronix Hackathon Registration Service from its current state through a production-ready nationwide system and ultimately to a multi-event, multi-tenant enterprise platform.

Phase 1 — Immediate Fixes

Timeline: Now → 2 weeks

- Move OTP storage from in-memory dict to Redis with 10-minute auto-expiring TTL
- Implement Celery + Redis for asynchronous background Excel file processing
- Implement Celery for non-blocking OTP email delivery via AWS SES
- Configure SlowAPI to use a Redis storage backend instead of in-process memory
- Add polling endpoint GET /api/jobs/{job_id} for frontend status checks
- Containerise the application with Gunicorn + Uvicorn multi-worker configuration
- Deploy to Kubernetes (EKS/GKE) with a minimum of 5 pods
- Configure Horizontal Pod Autoscaling (HPA) targeting 60% CPU utilisation
- Place an AWS Application Load Balancer with health checks in front of the cluster

Phase 2 — Production Hardening

Timeline: 2–6 weeks

- Migrate payment proof image storage from MongoDB Base64 to AWS S3
- Implement S3 Presigned URL pattern for direct browser-to-S3 Excel uploads
- Move the entire static/ directory to S3 and configure CloudFront CDN distribution
- Upgrade to MongoDB Atlas M30 or higher with a 3-node replica set
- Add missing MongoDB indexes on status and createdAt fields
- Configure explicit Motor connection pool limits (maxPoolSize=10 per process)
- Set up Prometheus + Grafana for real-time observability and dashboards
- Configure Alertmanager with on-call escalation for critical metrics
- Implement Redis caching for Hackathon settings fetched from MongoDB (5 min TTL)

Phase 3 — Resilience & Observability

Timeline: 6–12 weeks

- Deploy KEDA (Kubernetes Event-Driven Autoscaling) for queue-depth-based Celery worker auto-scaling during upload surges
- Implement circuit breakers around AWS SES and all external API dependencies
- Set up Sentry for application exception tracking with full stack trace context
- Configure Cloudflare WAF rules tailored to the registration application's traffic
- Implement a user-visible registration queue with real-time position tracking to handle thundering-herd deadline events gracefully
- Set up centralised log aggregation with Grafana Loki across all pods and workers

- Conduct a full 30,000 VU load test against the new production architecture
- Implement blue/green or canary deployment strategy via Kubernetes
- Add structured JSON logging with correlation IDs for distributed tracing

Phase 4 — Platform Evolution

Timeline: 3–6 months

- Multi-event support: allow administrators to create and manage multiple hackathons from a single platform deployment
- Multi-tenancy: support multiple organising institutions with isolated data spaces
- Self-service admin portal: allow event coordinators to configure registration amounts, domains, deadlines, and team constraints without engineering involvement
- Real-time dashboard: live registration counts, payment status, and geographic distribution using WebSockets or Server-Sent Events (SSE)
- Advanced analytics: reporting on registration trends, domain popularity, college participation rates, and payment compliance
- API gateway: introduce Kong or AWS API Gateway for centralised authentication, request transformation, and developer-facing API documentation
- Automated certificate generation: background Celery task generates participation certificates as PDF using ReportLab upon payment confirmation
- Bulk import optimisation: streaming Excel parse using openpyxl row-by-row iteration to handle files exceeding 100,000 rows without memory pressure
- CDN-based document delivery: store and serve generated certificates and reports directly from CloudFront with signed URLs for secure access

Phase 5 — Enterprise Scale

Timeline: 6–12 months

- Multi-region deployment: replicate the architecture across multiple AWS regions (e.g. ap-south-1 Mumbai + ap-south-2 Hyderabad) with latency-based routing
- Database sharding: shard the MongoDB registrations collection by state or region to distribute write load across multiple database shards
- GraphQL API layer: add a GraphQL endpoint alongside REST for flexible frontend querying and reduced over-fetching
- Automated infrastructure as code: full Terraform or Pulumi IaC for reproducible environment provisioning across staging, pre-production, and production
- Service mesh: deploy Istio or Linkerd to enforce mutual TLS between all services, provide automatic circuit breaking, and enable fine-grained traffic management
- SIEM integration: stream all security-relevant logs to a Security Information and Event Management system for compliance and audit requirements
- Capacity forecasting: ML-based traffic prediction to pre-warm the cluster 2 hours before a predicted deadline surge rather than reacting after it begins

6. Nationwide Traffic Model

6.1 Traffic Demand Calculation

Parameter	Value	Basis
Total eligible colleges in India	~50,000	Government + private engineering colleges
Expected registrations	~30,000	60% adoption rate assumption
Daily active users (normal period)	~5,000	~17% of registered users active daily
Peak-hour users on deadline day	~30,000	All remaining institutions rush simultaneously
Requests per active user per minute	3–5	Dashboard, upload, payment actions
Required peak RPS (minimum)	1,500–2,500 RPS	30,000 users × 3 req/min ÷ 60
Required peak RPS with 2× safety margin	5,000 RPS	Recommended design target

6.2 Registration Deadline Day Timeline

The most dangerous traffic event is not a gradual increase but the last-minute deadline rush. The following timeline shows how the auto-scaling system responds:

Time	Concurrent Users	Active Pods	System State
10:00 AM	~500	5	Normal. CPU 20%. Off-peak.
2:00 PM	~3,000	15	HPA triggers. Scaling starts.
5:00 PM	~12,000	25	Heavy traffic. Redis absorbing all OTPs and rate limits.
10:00 PM	~22,000	40	Near peak. Celery workers processing Excel backlog.
11:45 PM	~30,000	50	Maximum load. All SLOs maintained.
12:00 AM	~1,000	5 (scaling down)	Deadline passed. HPA scales back within 10 min.

6.3 Cost Estimation

Component	AWS Service	Est. Monthly Cost (USD)
Kubernetes cluster	AWS EKS (control plane)	~$73
Application pods (5–50)	EC2 t3.large on-demand	$300–$1,500
Celery worker pods (2–20)	EC2 t3.medium on-demand	$80–$400
Load Balancer	AWS ALB	$25–$100
MongoDB database	MongoDB Atlas M40 tier	~$240
Redis cache cluster	AWS ElastiCache r6g.large	~$180

Object storage	AWS S3 (uploads + static)	$30–$100
Content Delivery Network	AWS CloudFront	$50–$200
Email delivery	AWS SES	~$10 / 100,000 emails
Observability	Grafana Cloud free tier	$0–$50
TOTAL (off-peak / baseline)		~$900/month
TOTAL (during registration window)		~$2,500/month

7. Scalability Considerations & Edge Cases

Thundering Herd at Deadline

When thousands of colleges attempt registration in the final minutes simultaneously, every system layer receives maximum load at once. Without mitigation, this causes resource exhaustion faster than auto-scaling can respond.

Mitigations: Implement a visible queue with position tracking. Pre-warm the cluster to 30 pods 2 hours before the deadline. Apply Cloudflare Under Attack Mode to challenge suspicious volumetric traffic during deadline hours.

OTP Distributed Race Condition

Currently, if Pod A generates an OTP and stores it in its in-memory dict, but the user's subsequent verification request routes to Pod B, the verification fails because Pod B's dict has no record of the OTP. This will affect every user on multi-pod deployments.

Fix: Move all OTP storage to Redis immediately (Phase 1 item).

Database Write Contention

MongoDB enforces uniqueness across 7 fields in nodal_registrations. Under simultaneous high-concurrency inserts, each write requires checking all 7 unique indexes, creating contention.

Mitigation: The index strategy is correct. Upgrade to MongoDB Atlas M40+ for adequate IOPS. Return clear, user-friendly error messages on constraint violations.

Celery Worker Backlog

If 5,000 colleges upload Excel files within one hour, thousands of tasks queue in Redis. A fixed number of Celery workers will process these at a fixed rate, creating a growing backlog and poor user experience.

Mitigation: Deploy KEDA on Kubernetes to auto-scale Celery workers based on Redis queue depth. Scale from 2 to 50 worker pods within 30 seconds of a queue surge.

External Dependency Failures

AWS SES, the MongoDB Atlas connection, or Redis can experience temporary unavailability. Without protection, a slow external service causes cascading failures propagating back to the user-facing API.

Mitigation: Implement circuit breakers (circuitbreaker Python library) around all external calls. Celery tasks auto-retry with exponential backoff up to 3 attempts.

Memory Leak Under Soak

Long-running soak tests (30+ minutes) can reveal memory leaks in async Python applications, particularly in large object accumulation within coroutine closures or in Motor's connection pool under sustained load.

Mitigation: Run a 1-hour soak test against the new architecture. Monitor RSS memory per pod via Prometheus. Configure pod memory limits and restart policies.

8. Final Verdict & Prioritised Action Plan

8.1 Engineering Verdict

Verdict Item	Value
Current sustainable capacity	~100 concurrent users / ~50 RPS
Recommended safe operating capacity (after Phase 1 fixes)	~2,000 users / ~500 RPS (5 pods)
Maximum capacity after full implementation	~30,000+ users / ~5,500 RPS (50 pods)
Primary bottleneck	Excel parsing blocks the async event loop (synchronous Pandas)
Secondary bottleneck	Synchronous SMTP OTP delivery / In-memory rate limiting
Critical broken feature (multi-node)	In-memory OTP storage fails with multiple pods
Production readiness — current	NOT READY
Production readiness — after Phase 1 + Phase 2	READY WITH CONDITIONS
Production readiness — after Phase 3	FULLY READY

8.2 Prioritised Action Checklist

Phase 1 — Critical Prerequisites (2 weeks)

- ■ Move OTP storage from in-memory otp_store to Redis with 10-minute TTL
- ■ Implement Celery + Redis for asynchronous background Excel file processing
- ■ Implement Celery for non-blocking OTP email delivery
- ■ Configure SlowAPI to use Redis storage backend
- ■ Add GET /api/jobs/{job_id} polling endpoint for frontend status checks
- ■ Containerise with Gunicorn + 4 Uvicorn workers per pod
- ■ Deploy to Kubernetes with minimum 5 pods and HPA targeting 60% CPU
- ■ Place AWS Application Load Balancer in front of the cluster

Phase 2 — Production Hardening (6 weeks)

- ■ Migrate payment proof images from MongoDB Base64 to AWS S3
- ■ Implement S3 Presigned URL pattern for direct browser-to-S3 Excel uploads
- ■ Move static/ to S3 and configure CloudFront CDN distribution
- ■ Upgrade to MongoDB Atlas M40 with 3-node replica set
- ■ Add missing MongoDB indexes on status and createdAt fields
- ■ Configure Motor connection pool: maxPoolSize=10 per process
- ■ Set up Prometheus + Grafana + Alertmanager observability stack

Phase 3 — Resilience & Observability (12 weeks)

- ■ Deploy KEDA for queue-depth-based Celery worker auto-scaling
- ■ Implement circuit breakers around all external API dependencies

- ■ Set up Sentry for exception tracking
- ■ Configure Cloudflare WAF rules for the application
- ■ Implement user-visible registration queue for deadline events
- ■ Set up Grafana Loki for centralised log aggregation
- ■ Conduct full 30,000 VU load test against the new architecture

The Spheronix Hackathon Registration Service has well-structured, correct business logic. By resolving the four architectural issues identified in this report — synchronous Excel processing, synchronous email delivery, in-memory shared state, and single-node deployment — the same application codebase will reliably serve 30,000 concurrent users with automatic failover, auto-scaling, and full engineering observability.